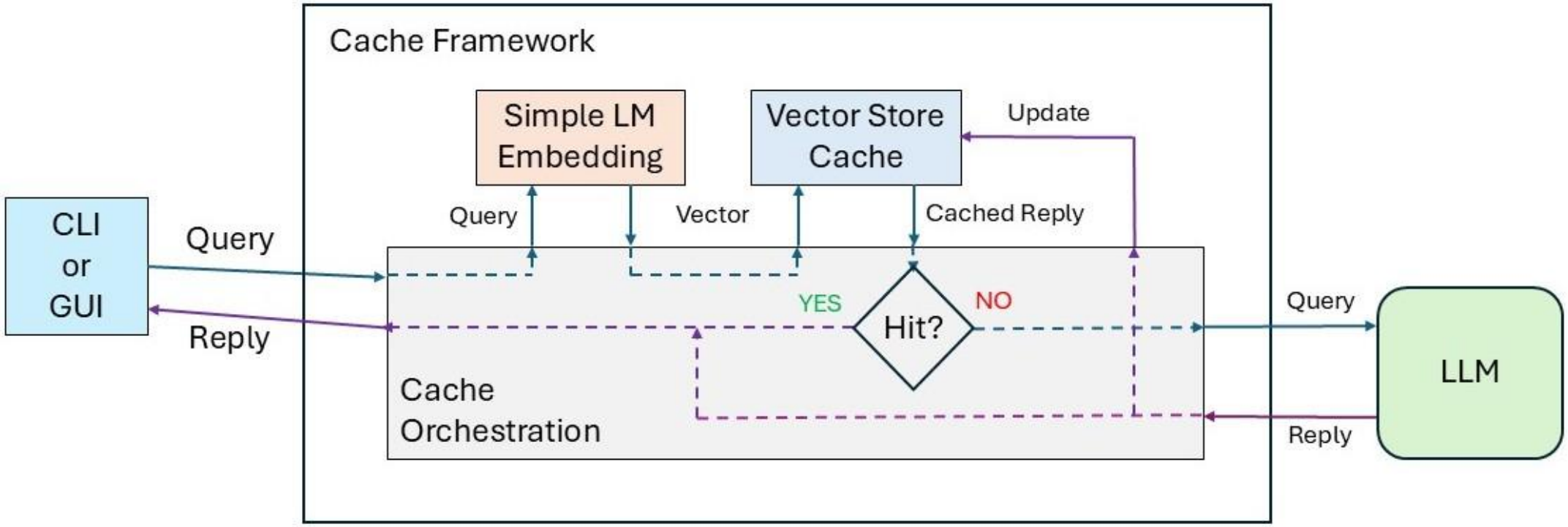
LLM Semantic Cache

Tomer Eliel · Adi Volfman

Final high-level design overview

Map the original cache framework to the final service architecture



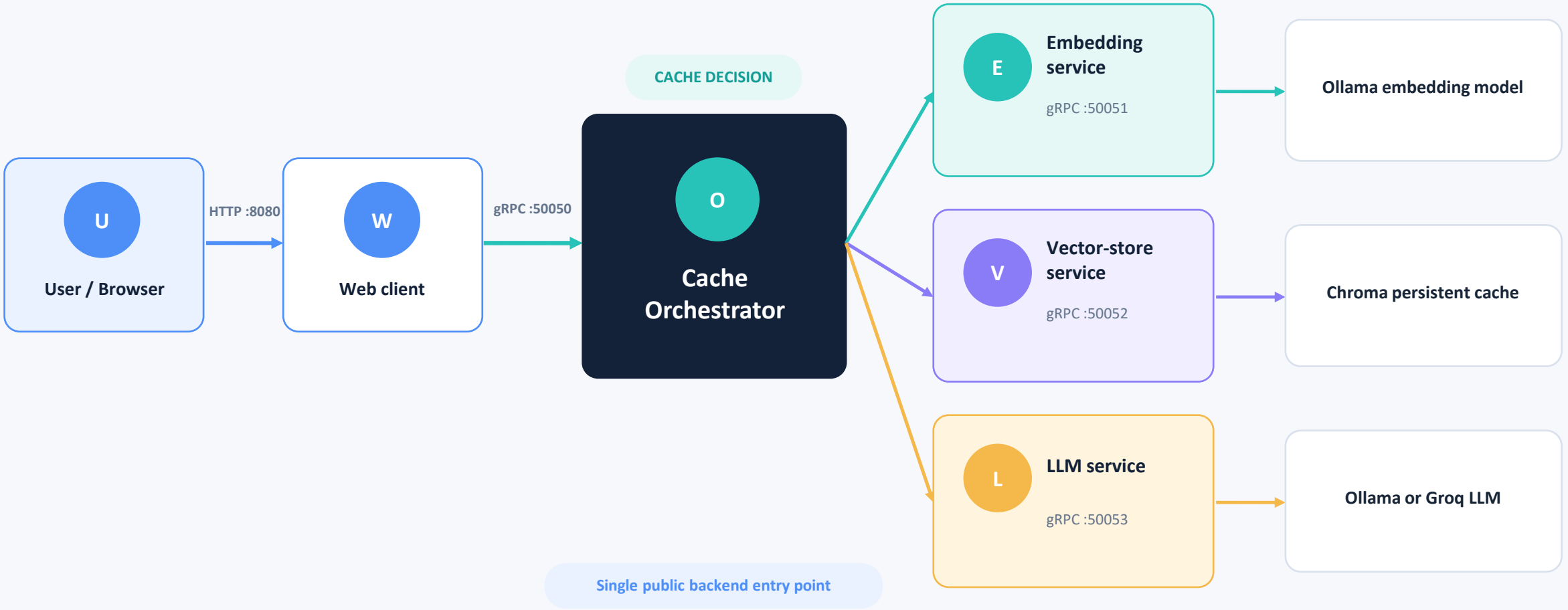
CLI / GUI → Web client
HTTP :8080

Cache orchestration
Orchestrator + gRPC clients

Pluggable backends
Embedding, vector store, LLM

Core invariant: every request embeds and searches first, the LLM is called only on a cache miss.

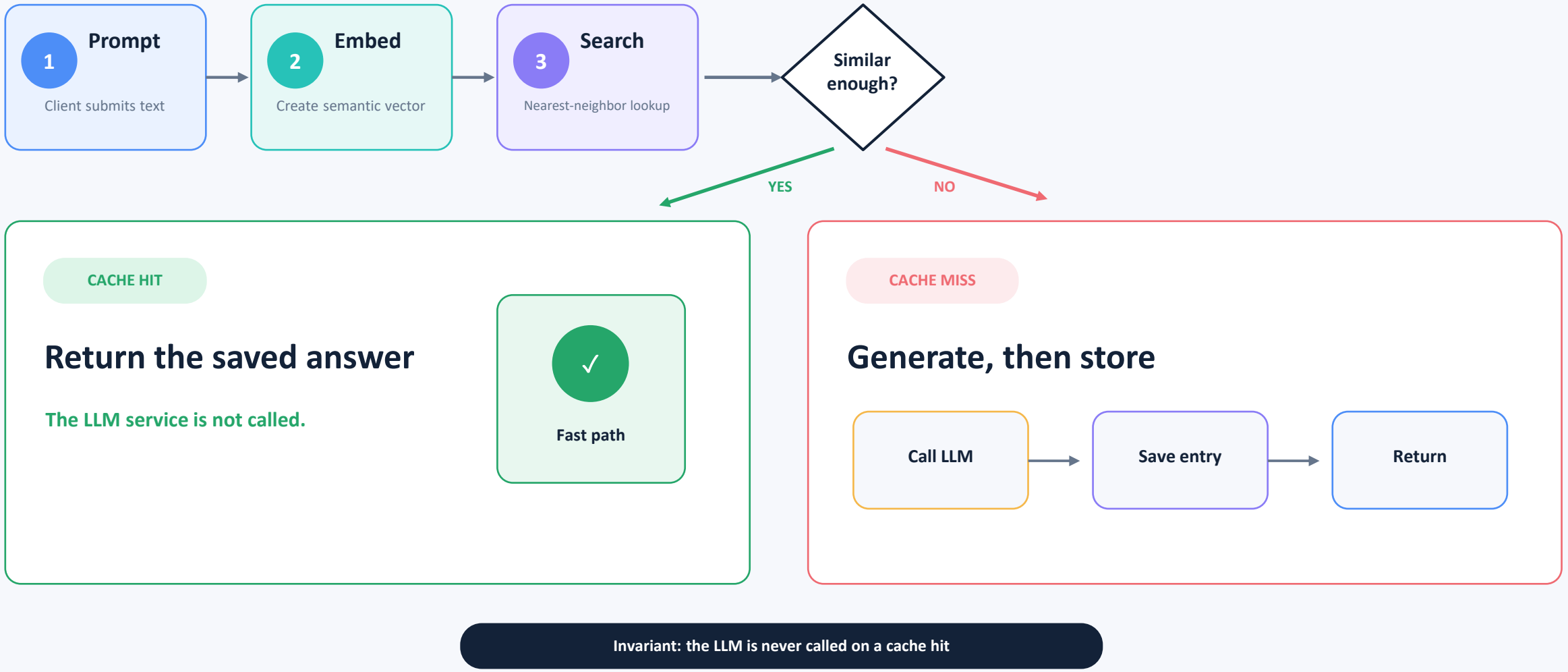
System architecture



Only the web client is exposed to the user. Provider services remain independently replaceable and deployable.

One request, two possible paths

The cache lookup is always performed before the expensive generation step.



Invariant: the LLM is never called on a cache hit

The key design choice: dependency inversion

Remote services look local to the cache algorithm because gRPC clients implement the same interfaces.

BUSINESS LOGIC

CacheOrchestrator

Depends only on three capabilities:

- IEmbedder
- IVectorStore
- ILLMProvider

No provider or transport details inside the algorithm.

same contract →

ADAPTERS

EmbeddingGrpcClient

or

OllamaEmbedder

VectorStoreGrpcClient

or

ChromaVectorStore

LLMGrpcClient

or

Ollama / Groq

01 Replace providers

Switch Ollama, Groq, Chroma, or in-memory implementations without rewriting the workflow.

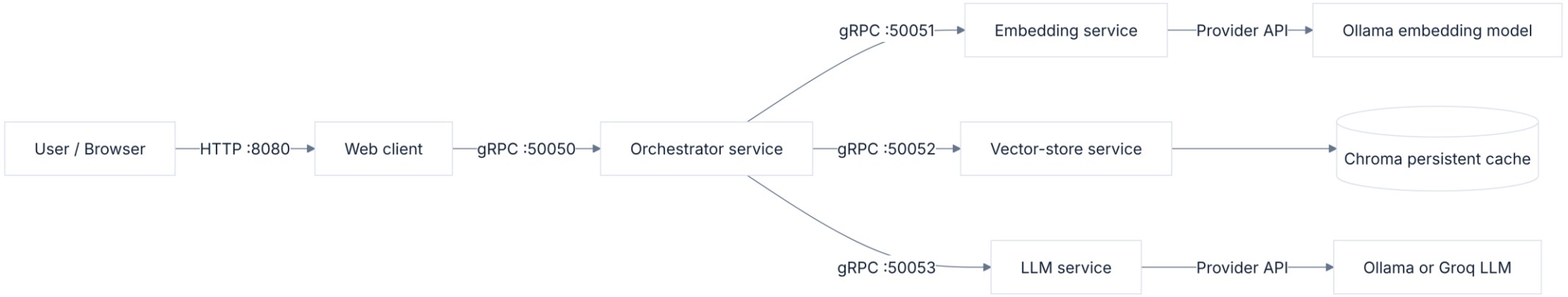
02 Switch deployment mode

The orchestrator can receive local objects or remote gRPC client adapters.

03 Test in isolation

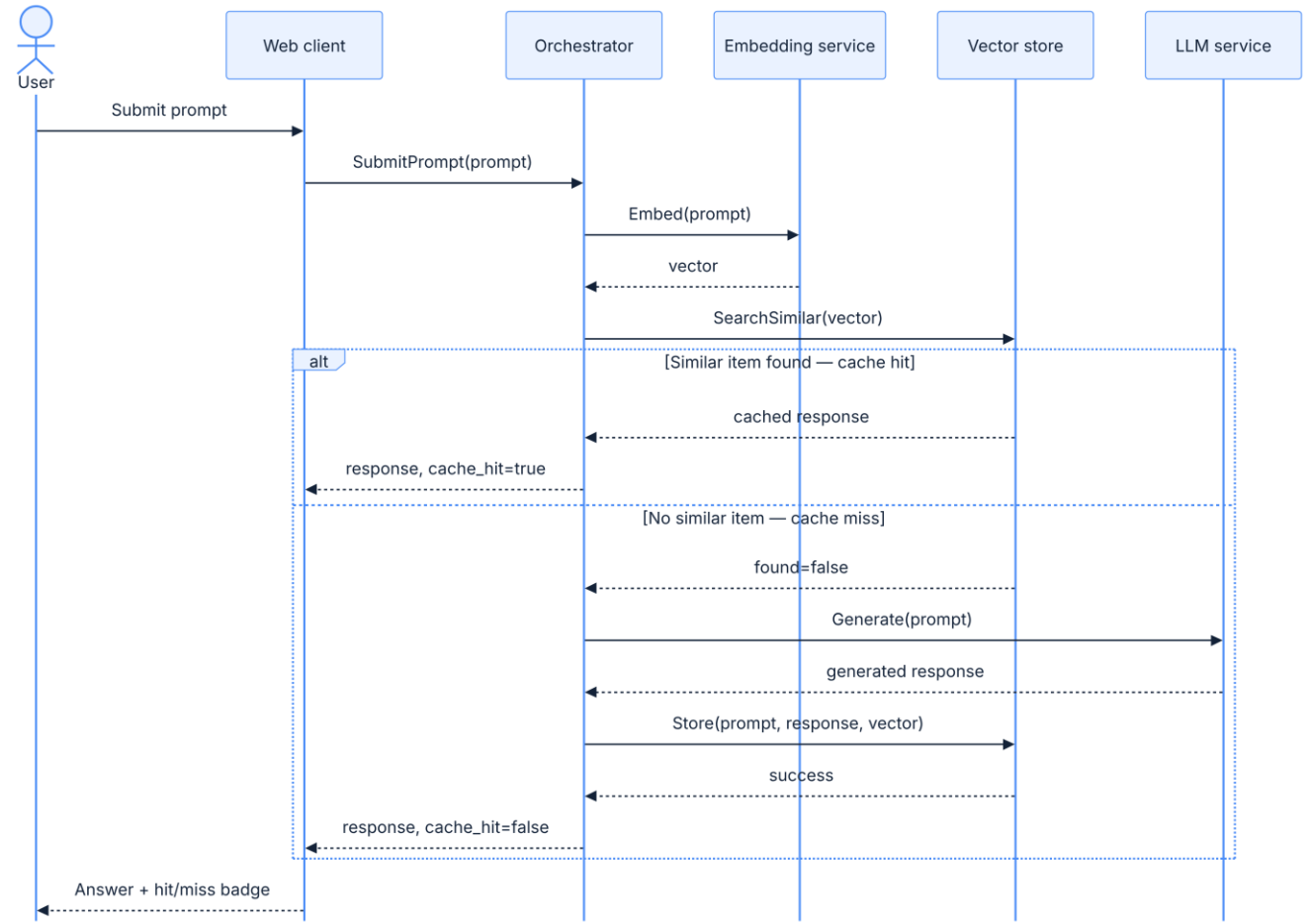
Stubs and spies verify hit/miss behavior without real network or model dependencies.

Data Flow Architecture Diagram



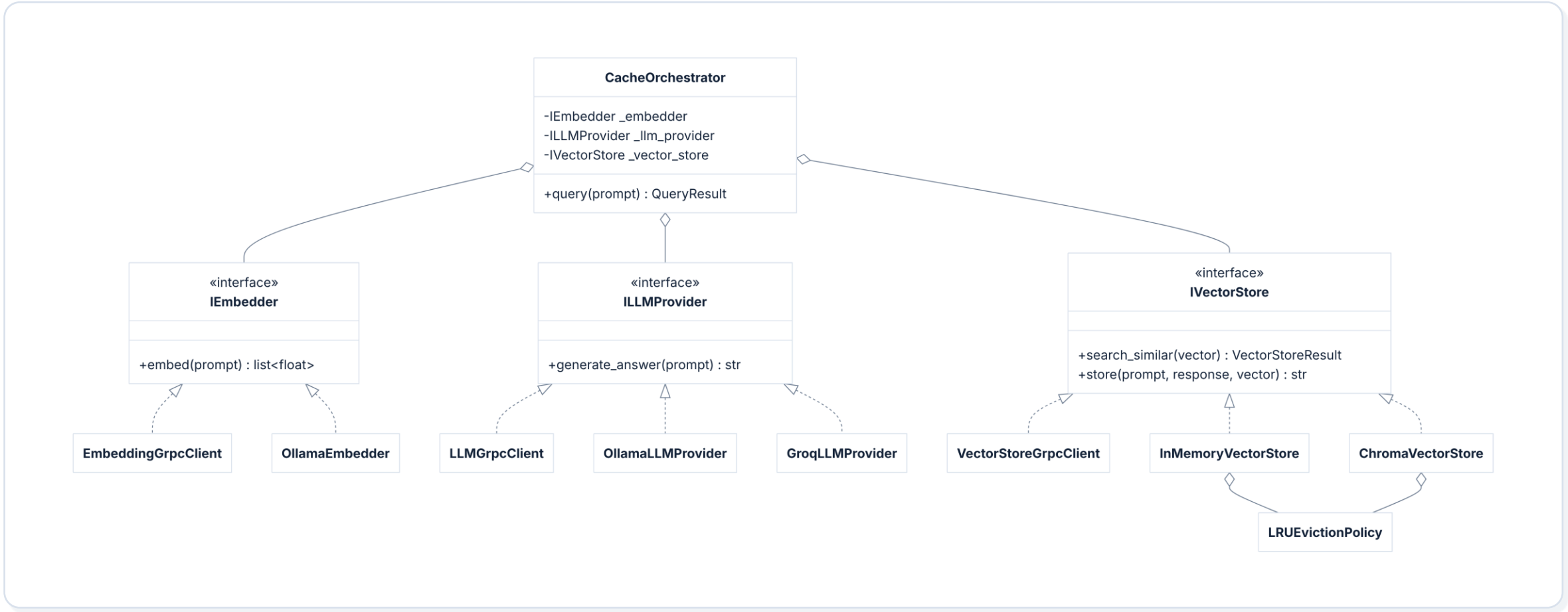
Main Query Flow – Sequence Diagram

Sequence diagram: every request embeds first, searches the cache, then either returns a hit or generates and stores a miss



Class Diagram

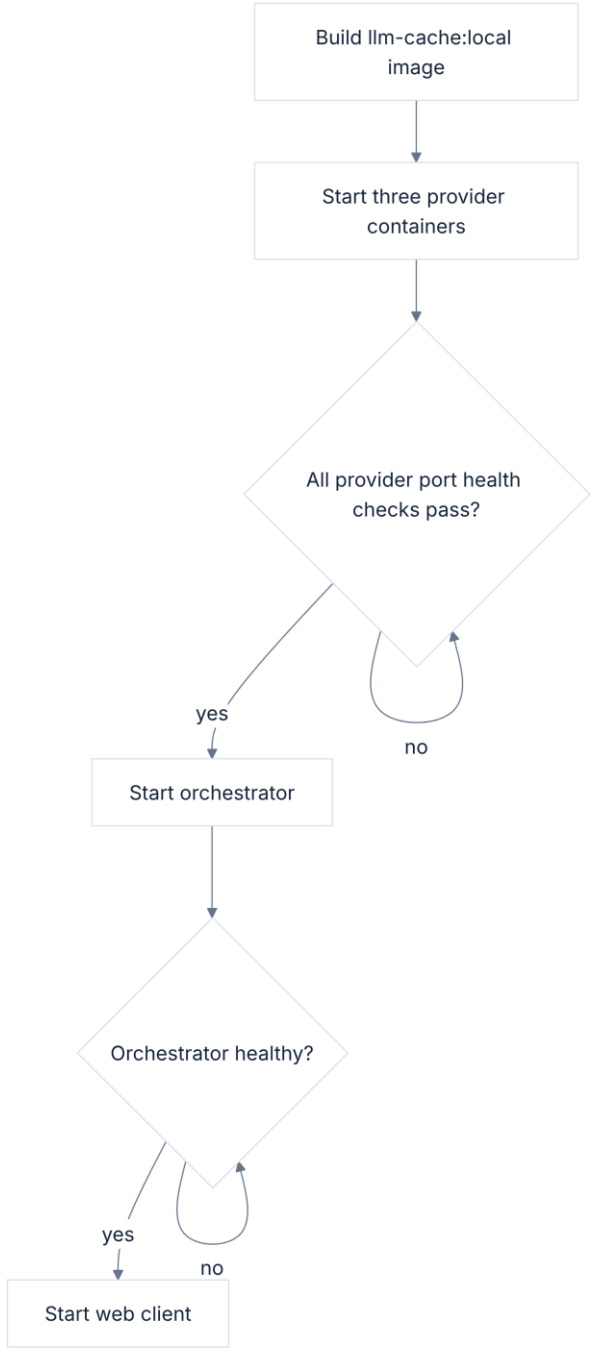
The orchestrator depends on interfaces local providers and gRPC clients are interchangeable implementations



This is the dependency-inversion story in class form.

Docker Startup Flow

Compose starts the provider services first, waits for readiness, then starts the orchestrator and web client



What changed since the alpha demo

Final runtime

- Five-process distributed stack
- gRPC interfaces between services
- Docker Compose health-gated startup

Final providers

- Ollama embeddings + LLM
- Optional Groq LLM
- Chroma or in-memory vector store

Final UX

- Web UI
- CLI loop
- hit/miss status

Engineering additions

- Cache persistence is now configurable
- Central JSON config + CLI overrides
- Provider registry + factories
- Test doubles for deterministic validation

Operational additions

- Structured logs with request_id
- Health checks and setup checks
- Trace analysis for real prompt datasets

Submission additions

- README, User Guide, Developer Guide
- Docker / logging / distance docs
- Tests + ruff + docker compose validation

Final demo script- what changed since alpha presentation

1. Start stack

ollama serve + model pull

docker compose up --build

2. Open UI

http://127.0.0.1:8080

Health gate enables prompt box

3. Fresh response

Submit first prompt

Show cache_hit=false / Fresh response

4. Cache hit

Submit same or close prompt

Show cache_hit=true and no LLM call

5. Trace logs

docker compose logs

Filter by request_id across services

Developer guide: module map

The repo contains a developer guide and module-level documentation

Core services

src/llm_cache/orchestrator — cache decision + public gRPC API

src/llm_cache/embedding — IEmbedder, Ollama, embedding gRPC

src/llm_cache/vector_store — IVectorStore, Chroma, memory, LRU, gRPC

src/llm_cache/llm — ILLMProvider, Ollama/Groq, LLM gRPC

Shared infrastructure

src/llm_cache/config — dataclasses, JSON config, CLI parsing, registry

src/llm_cache/factories — provider construction from config

src/llm_cache/health — setup checks and readiness helpers

src/llm_cache/web + cli — user-facing clients

Guides and docs

docs/USER_GUIDE.md — install, configure, run, troubleshoot

docs/DEVELOPER_GUIDE.md — HLD, files, tests, extension rules

docs/DOCKER.md / LOGGING.md — deployment and request tracing

docs/VECTOR_STORE_DISTANCE.md — threshold and distance semantics

Project summary

Reflection

What went well

Clean dependency inversion

Same logic works local or gRPC

Multiple providers without rewriting orchestrator

Deterministic tests and demos

Main difficulties

Learning about each component we used, learning about how they communicate together.

gRPC wiring and protobuf generation

Similarity threshold semantics: cosine vs Chroma distance

Potential extensions

Admin page for cache inspection

Live metrics dashboard: visualize cache hit rate, response latency, LLM calls avoided, and cache usage over time.

Adaptive similarity thresholds- instead of one fixed threshold, choose thresholds based on prompt type, provider, or confidence.

AI tools used

ChatGPT/Codex for design review, debugging, docs, and tests

AI helped speed iteration but all code was reviewed and validated

Useful for refactoring, less reliable when context was missing

Final validation and quality gates

Automated checks

uv run ruff format --check .

uv run ruff check .

uv run pytest

docker compose config

docker compose build

Test coverage areas

Orchestrator hit/miss behavior

Embedding, LLM, and vector-store gRPC services

Factories and runtime config

Web/CLI clients, health, tracing, demos

Known limitations

In-memory cache is not persistent and it is not configurable

Live demos need Ollama models or Groq credentials